EPM – EFFICIENT PREDICTIVE MULTIMODAL GENERATIVE MODEL

A Reproducible Platform for Training, Benchmarking and Comparing Machine Learning Algorithms

A Project Report

Submitted in partial fulfillment of the requirements
for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

BY

PRANSHU

(ROLL NO.: _____)



BIRLA INSTITUTE OF TECHNOLOGY
MESRA-835215, RANCHI
2026

APPROVAL OF THE GUIDE

Recommended that the project report entitled **"EPM – Efficient Predictive Multimodal Generative Model: A Reproducible Platform for Training, Benchmarking and Comparing Machine Learning Algorithms"** presented by **Pranshu** under my supervision and guidance be accepted as fulfilling this part of the requirements for the award of Degree of **Bachelor of Technology in Computer Science and Engineering**. To the best of my knowledge, the content of this report did not form a basis for the award of any previous degree to anyone else.

Date: _____

_____

Name of the Guide
Designation
Dept. of Computer Science and Engineering
Birla Institute of Technology
Mesra, Ranchi

DECLARATION CERTIFICATE

I certify that

a) The work contained in the report is original and has been done by myself under the general supervision of my supervisor.

b) The work has not been submitted to any other Institute for any other degree or diploma.

c) I have followed the guidelines provided by the Institute in writing the report.

d) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

e) Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references.

f) Whenever I have quoted written materials from other sources, I have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Pranshu

(Roll Number: _____)

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this report entitled "EPM – Efficient Predictive Multimodal Generative Model: A Reproducible Platform for Training, Benchmarking and Comparing Machine Learning Algorithms" is carried out by Pranshu (Roll No. _____) and is approved for the degree of Bachelor of Technology in Computer Science and Engineering of Birla Institute of Technology, Mesra, Ranchi.

_____ _____

Internal Examiner **External Examiner**

_____

(Chairman)
Head of Department

Date: _____

Place: Ranchi

ABSTRACT

Machine-learning models are often compared unfairly: each model is trained on a different split, preprocessing is fitted on the whole dataset, and accuracy is used for every kind of problem. This project presents **EPM (Efficient Predictive Multimodal Generative Model)**, a platform that trains, evaluates and compares machine-learning algorithms under identical, leakage-free and reproducible conditions.

EPM was developed in fourteen stages, from mathematical foundations and from-scratch implementations to deep learning, generative models (VAE, diffusion, flow matching) and model compression. The integration work connects these into one system: a registry of **20 documented public and synthetic datasets**, **44 algorithm configurations** across regression, classification, clustering, forecasting and anomaly detection, automatic compatibility checks, preprocessing pipelines fitted on training data only, and a benchmark engine that reports task-specific metrics, cross-validation spread, latency, model size and a Pareto front.

Users can run experiments through a web-based **Experiment Lab**, the command line, or an offline **chatbot** that understands plain-English requests such as "classify iris with random forest" using rule-based and TF-IDF intent recognition and a conversation state machine. The platform is verified by **502 automated tests (100% passing)**.

Measured benchmarks show, for example, that simple models often match complex ones once cross-validation spread is considered, and that a naive baseline beat deep forecasters on short-horizon telemetry. A controlled study (Stage 13) found that the same diffusion forecaster sampled with 10 instead of 100 steps is **7.66× faster** while its error stays within the full-step confidence interval.

Keywords: machine learning, benchmarking, data leakage, reproducibility, diffusion models, flow matching, chatbot.

Source code: https://github.com/webex042/effective-prediction-model

Documentation:
https://github.com/webex042/effective-prediction-model/tree/main/docs

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, [Name of the Guide], Department of Computer Science and Engineering, Birla Institute of Technology, Mesra, for valuable guidance, encouragement and constructive feedback throughout this project.

I am thankful to the Head of the Department and all faculty members of the Department of Computer Science and Engineering for providing the facilities and a supportive learning environment.

I also thank the open-source communities behind scikit-learn, PyTorch, XGBoost, the UCI Machine Learning Repository, OpenML and the Numenta Anomaly Benchmark, whose tools and datasets made this work possible.

Finally, I am grateful to my family and friends for their constant support and motivation.

DATE: _____ **Pranshu**

(Roll Number: _____)

CONTENTS

ABSTRACT ..... **v**
ACKNOWLEDGEMENT ..... **vi**
LIST OF FIGURES ..... **viii**
LIST OF TABLES ..... **viii**
1 INTRODUCTION **1**
1.1 OVERVIEW 1
1.2 PROBLEM STATEMENT 1
1.3 MOTIVATION 1
1.4 OBJECTIVES 2
1.5 PROJECT LINKS 2
1.6 ORGANIZATION OF THE REPORT 2
2 BACKGROUND AND LITERATURE REVIEW **3**
2.1 MACHINE LEARNING TASK FAMILIES 3
2.2 FAIR EVALUATION AND DATA LEAKAGE 3
2.3 GENERATIVE MODELS 4
2.4 MODEL COMPRESSION AND CALIBRATION 4
2.5 RESEARCH GAP 4
3 SYSTEM DESIGN AND ARCHITECTURE **5**
3.1 DEVELOPMENT STAGES 5
3.2 SYSTEM ARCHITECTURE 5
3.3 TECHNOLOGY STACK 6
3.4 PROJECT STRUCTURE 6
4 DATASET REGISTRY AND PREPROCESSING **7**
4.1 THE DATASET REGISTRY 7
4.2 INTEGRATED DATASETS 7
4.3 SUPPORTED ALGORITHMS 8
4.4 INTELLIGENT DATASET SELECTION 8
4.5 LEAKAGE-FREE PREPROCESSING 8
5 BENCHMARKING AND EVALUATION **10**
5.1 FAIR BENCHMARKING PROCEDURE 10
5.2 EVALUATION METRICS 10
5.3 CLASSIFICATION RESULTS 10
5.4 REGRESSION RESULTS 11
5.5 RESULTS ON OTHER TASKS 12
6 USER INTERFACE AND EPM CHATBOT **13**
6.1 WEB DASHBOARD 13
6.2 EXPERIMENT LAB 13
6.3 CHATBOT DESIGN 14
6.4 SAFETY GUARDRAILS 15
7 TESTING AND RESEARCH RESULTS **16**
7.1 AUTOMATED TESTING 16
7.2 STAGE 13: REDUCED-STEP GENERATIVE FORECASTING 16
7.3 FINDINGS AND LIMITATIONS 17
8 CONCLUSION AND FUTURE SCOPE OF WORK **18**
8.1 CONCLUSION 18
8.2 FUTURE SCOPE 18
APPENDIX A: HOW TO RUN THE PROJECT **19**
REFERENCES **20**

LIST OF FIGURES

Figure 3.1 Layered architecture of the EPM platform.....6
Figure 4.1 Experiment pipeline; preprocessing (step 4) is fitted on training rows only.....8
Figure 5.1 Experiment Lab result view with leaderboard, charts and confusion matrix.....11
Figure 6.1 EPM dashboard overview page with the research question and verified results.....13
Figure 6.2 Experiment Lab: task tabs, dataset catalogue and algorithm finder.....14
Figure 6.3 Conversation state machine of the EPM chatbot.....15
Figure 6.4 Chat panel running a classification benchmark on the Wine dataset.....15
Figure 7.1 Accuracy and latency of one diffusion model at different step counts.....17

LIST OF TABLES

Table 1.1 Project links.....2
Table 2.1 Key terms used in this report.....3
Table 3.1 Development stages of EPM.....5
Table 3.2 Technology stack.....6
Table 4.1 Datasets integrated into EPM (rows × features).....7
Table 4.2 Algorithms available for each task.....8
Table 4.3 Preprocessing steps and when they are applied.....9
Table 5.1 Metrics used for each task.....10
Table 5.2 Breast Cancer Wisconsin – classification benchmark.....10
Table 5.3 Diabetes progression – regression benchmark.....11
Table 5.4 Best models on clustering, anomaly, forecasting and image tasks.....12
Table 6.1 Chatbot intents (25 in total).....14
Table 7.1 Summary of the automated test suite.....16
Table 7.2 Stage 13 efficiency analysis (reference: 100 steps).....16
Table A.1 Useful example chat messages.....19

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

EPM (**Efficient Predictive Multimodal Generative Model**) is a machine-learning experimentation platform. It lets a user pick a prediction task, choose a suitable public dataset, select compatible algorithms and run a *fair* benchmark, all from a web page, a terminal command or a plain-English chat message. Every run uses the same data split and random seed for every model, fits preprocessing on training data only, reports task-appropriate metrics together with speed and model size, and saves a reproducible record of the experiment.

The project was built in fourteen stages (Stage 0 to Stage 13). The early stages implement the mathematics and the core algorithms from scratch; the middle stages add deep learning, time-series and multimodal models; the later stages study generative models (VAE, diffusion, flow matching) and model compression. The final integration work, described in this report, connects all of these into one platform through a dataset registry, leakage-free preprocessing pipelines, a benchmarking engine, an interactive Experiment Lab and a rule-based chatbot.

1.2 PROBLEM STATEMENT

Students and practitioners often compare machine-learning models in ways that are not fair or not repeatable. Common problems include training each model on a different random split, scaling the data before splitting (so information from the test set leaks into training), using accuracy for every task, ignoring inference speed and memory, and forgetting which settings produced a result. Algorithms are also frequently applied to datasets they were not designed for, such as shuffling a time series or training a clustering model with the labels.

The problem addressed by EPM is therefore: build a single platform in which any supported algorithm can be trained and evaluated on a suitable, documented dataset under identical and leakage-free conditions, with results that are easy to understand and can be reproduced later.

1.3 MOTIVATION

- **Fair comparison:** a ranking is only meaningful if all models see exactly the same training and test rows.
- **Correct evaluation:** regression, classification, clustering, forecasting and anomaly detection each need different metrics and different splitting rules.
- **Efficiency matters:** the most accurate model is not always the best choice when latency, size and interpretability are considered.
- **Accessibility:** a newcomer should be able to run a correct experiment without writing code, by clicking or by typing a sentence.

1.4 OBJECTIVES

1. Audit the existing EPM code and identify every implemented algorithm and dataset.
2. Build a central dataset registry with source, licence, size, task and preprocessing metadata.
3. Recommend compatible datasets for an algorithm and compatible algorithms for a dataset.
4. Implement algorithm-aware preprocessing that is fitted on training data only.
5. Create a fair benchmarking pipeline with task-specific metrics, latency, model size and a Pareto front.
6. Integrate everything into the web interface (Experiment Lab) and a natural-language chatbot.
7. Verify the work with automated tests and document how to run and extend the platform.

1.5 PROJECT LINKS

The complete source code, documentation and experiment records are publicly available. The links below can be opened directly from this document.

Table 1.1: Project links

Resource	Link	What you will find
GitHub repository	github.com/webex042/effective-prediction-model	Full source code, tests, web dashboard and experiment records
Documentation	github.com/webex042/effective-prediction-model/tree/main/docs	17 chapter-style guides (docs/01 to docs/16) and the Stage 13 research report
Quick-start guide	github.com/webex042/effective-prediction-model#readme	README with installation steps, commands and project structure

1.6 ORGANIZATION OF THE REPORT

Chapter 2 explains the background concepts in simple terms. Chapter 3 describes the system architecture and development stages. Chapter 4 covers the dataset registry and preprocessing. Chapter 5 explains the benchmarking method and presents measured results. Chapter 6 describes the user interface and the chatbot. Chapter 7 reports testing and the Stage 13 research experiment. Chapter 8 concludes the report and lists future work. Appendix A gives step-by-step instructions for running the project.

CHAPTER 2
BACKGROUND AND LITERATURE REVIEW

2.1 MACHINE LEARNING TASK FAMILIES

EPM supports five families of prediction problems. In **regression** the target is a number (for example, a house price). In **classification** it is a category (for example, benign or malignant). **Clustering** groups similar rows without using any labels. **Forecasting** predicts future values of a time series from its past. **Anomaly detection** finds rare, unusual observations. Classical algorithms for these tasks are provided by scikit-learn [1], deep-learning models are written in PyTorch [2], and gradient-boosted trees use XGBoost [3].

Table 2.1: Key terms used in this report

Term	Meaning in simple words
Training / validation / test split	Data used to fit a model, to tune it, and to give a final unbiased score. The test rows are never used for fitting.
Data leakage	Any information from validation or test data that reaches the training process, which makes scores look better than they really are [4].
Pipeline	A fixed chain of steps (impute, scale, encode, model) that is fitted once on training data and then applied unchanged to new data.
Cross-validation (CV)	Repeating training on several different folds of the training data to measure how stable a score is.
Baseline	A deliberately simple model (mean value, majority class, "tomorrow = today"). A useful model must beat it.
Pareto front	The set of models for which no other model is both more accurate and faster.
Latency	Time taken to make one prediction, reported in microseconds per sample including preprocessing.
NFE	Number of function evaluations: how many times a generative model's network is called to produce one sample.

2.2 FAIR EVALUATION AND DATA LEAKAGE

Kaufman et al. [4] showed that leakage is one of the most common reasons why published results fail in practice. Typical sources are fitting a scaler on the full dataset, removing duplicates after splitting, or using future values when forecasting. For time series, Hyndman and Koehler [5] recommend scale-free error measures such as MASE, which compare a model against a naive forecast and therefore reveal when a complex model is no better than "repeat the last value". For clustering, internal measures such as the silhouette score [6] judge cluster quality without labels. For anomaly detection, methods such as Isolation Forest [7] are commonly evaluated on the Numenta Anomaly Benchmark (NAB) [8], where precision-recall measures are preferred because anomalies are rare.

2.3 GENERATIVE MODELS

A **Variational Autoencoder (VAE)** [9] compresses data into a small latent space and learns to sample new data from it. **Diffusion models** [10] learn to remove noise step by step; DDIM [11] makes sampling faster by skipping steps. **Flow matching** [12], [13] learns a smooth path from noise to data and can produce good samples with very few steps. Transformers [14] use attention to model sequences. EPM implements all of these and studies a central question: can fewer sampling steps keep accuracy while making prediction much cheaper?

2.4 MODEL COMPRESSION AND CALIBRATION

Large models can be made smaller with **knowledge distillation** [15] (a small "student" network learns from a large "teacher"), **quantisation** (storing weights as 8-bit integers instead of 32-bit floats) and **pruning** (removing unimportant weights). A model should also be **calibrated**: when it says it is 90% sure, it should be right about 90% of the time [16]. EPM reports the Expected Calibration Error (ECE) and Brier score for this purpose.

2.5 RESEARCH GAP

Libraries provide excellent individual algorithms but leave the experimental protocol to the user. Public dataset repositories such as UCI [17] and OpenML [18] provide data but not guidance on which algorithms suit it. EPM fills this gap by combining a documented dataset catalogue, compatibility rules, leakage-free pipelines and an identical evaluation protocol in one tool that a beginner can use safely.

CHAPTER 3
SYSTEM DESIGN AND ARCHITECTURE

3.1 DEVELOPMENT STAGES

The project followed a strict progression rule: each stage was learned, implemented, tested, benchmarked and documented before the next stage started. Table 3.1 summarises the stages.

Table 3.1: Development stages of EPM

Stage	Topic	What was built
0	Environment	Project structure, configuration, logging, reproducible seeds
1	Mathematics	Linear algebra, calculus, probability and optimisation from scratch
2	Classical ML	Linear/logistic regression, k-NN, decision trees, SVM, boosting
3	Neural networks	Forward and backward propagation from scratch; PyTorch MLP
4	NLP	Tokenisation, embeddings and the attention mechanism
5	Time series	LSTM, GRU and Transformer forecasters for sensor data
6	Multimodal	Fusion of sensor readings with text maintenance logs
7	Generative models	Autoencoder, VAE, GAN concepts
8	Diffusion	DDPM and accelerated DDIM sampling
9	Diffusion for time series	Probabilistic trajectory forecasting with intervals
10	Flow matching	Optimal-transport conditional flow matching vs diffusion
11	Efficiency	Linear and tiled attention, complexity analysis
12	Compression	Distillation, INT8 quantisation, pruning
13	Research experiment	Reproducible study of reduced-step generative forecasting

3.2 SYSTEM ARCHITECTURE

EPM is organised in four layers (Figure 3.1). The user interface layer contains the web dashboard, the Experiment Lab, the chat panel and the command line. The API layer is a small local web server started by run.py; it queues training jobs so that the page never freezes and any job can be cancelled. The core services layer contains the chatbot, the benchmark runner, the preprocessing pipelines, the evaluation metrics, the dataset registry and the model modules from Stages 1 to 12. The storage layer keeps downloaded datasets, run records and an exported catalogue that the static (Vercel) version of the site can read.

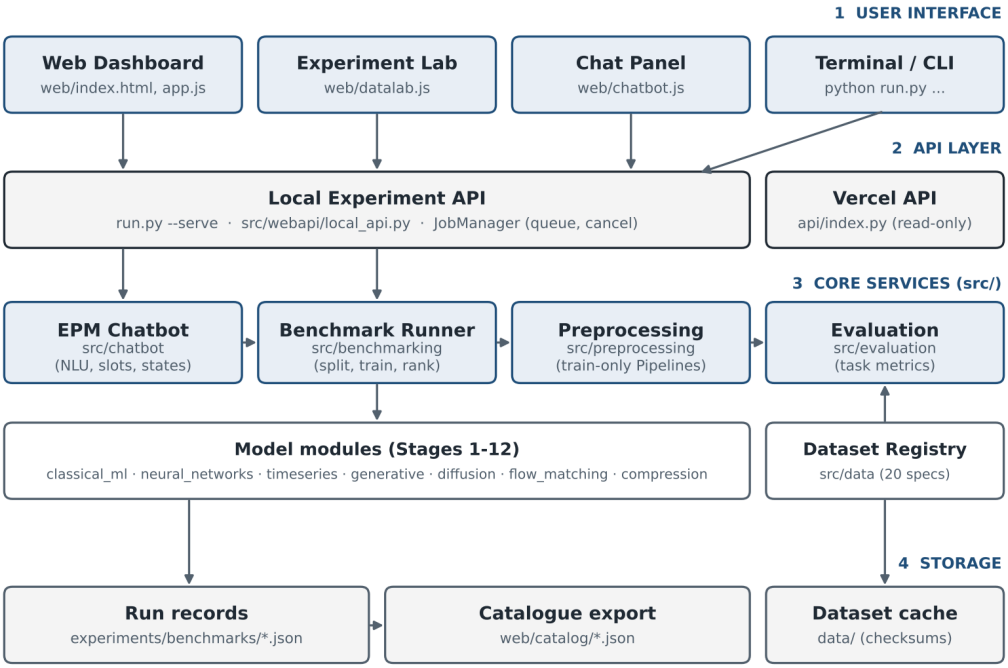


Figure 3.1: Layered architecture of the EPM platform

3.3 TECHNOLOGY STACK

Table 3.2: Technology stack

Layer	Technology	Why it was chosen
Language	Python 3.10 or newer (tested on 3.13)	Standard language for ML; rich ecosystem
Data handling	NumPy, pandas	Fast arrays and tabular data
Classical ML	scikit-learn, XGBoost	Reliable, well-tested algorithms and Pipelines
Deep learning	PyTorch	MLP, LSTM, GRU, Transformer, VAE, diffusion, flow matching
Web server	Python standard library (ThreadingHTTPServer)	No extra infrastructure needed; runs offline
Front end	HTML, CSS, plain JavaScript	Lightweight, no build step
Hosting	Vercel (static site + read-only API)	Free public preview of results
Testing	pytest	Automated verification of every module

3.4 PROJECT STRUCTURE

The most important folders are: `src/` (all Python modules, one sub-package per stage plus `data`, `preprocessing`, `benchmarking`, `evaluation`, `webapi` and `chatbot`), `web/` (the dashboard), `tests/` (23 test modules), `docs/` (the written guides), `experiments/` (saved runs) and `run.py` (the single entry point for every command).

CHAPTER 4
DATASET REGISTRY AND PREPROCESSING

4.1 THE DATASET REGISTRY

Every dataset is described once, in `src/data/registry.py`, by a record that stores its name, description, source and download URL, licence, supported tasks, feature and target columns, number of rows and features, missing-value information, approximate memory, recommended preprocessing, recommended metrics, splitting strategy and known caveats. Remote files are downloaded once, verified against the expected schema and cached locally; a failed download produces a clear message instead of a crash, and an unknown dataset is never silently replaced by a different one.

4.2 INTEGRATED DATASETS

Twenty datasets were integrated (Table 4.1). Small datasets load instantly and suit quick experiments; medium and large datasets give more realistic benchmarks. Large datasets must be explicitly allowed by the user.

Table 4.1: Datasets integrated into EPM (rows × features)

#	Dataset	Task(s)	Size	Source
1	Iris	Classification, clustering	150 × 4	scikit-learn / UCI
2	Wine Recognition	Classification, clustering	178 × 13	scikit-learn / UCI
3	UCI Seeds	Clustering, classification	210 × 7	UCI (id 236)
4	UCI Wholesale Customers	Clustering	440 × 6	UCI (id 292)
5	Diabetes (Efron et al.)	Regression	442 × 10	scikit-learn
6	Breast Cancer Wisconsin	Classification	569 × 30	scikit-learn / UCI
7	UCI Energy Efficiency	Regression	768 × 8	UCI (id 242)
8	EPM Sensor Regression	Regression	1,200 × 5	Synthetic (project)
9	EPM Equipment Failure	Classification	1,200 × 5	Synthetic (project)
10	Synthetic Gaussian Blobs	Clustering	1,500 × 6	Synthetic (scikit-learn)
11	Optical Digits (8×8)	Classification	1,797 × 64	scikit-learn / UCI
12	EPM Sensor Telemetry	Forecasting	2,000 × 5	Synthetic (project)
13	Synthetic Anomalies	Anomaly detection	2,000 × 8	Synthetic (fixed seed)
14	UCI Air Quality	Forecasting	9,357 × 12	UCI (id 360)
15	California Housing	Regression	20,640 × 8	StatLib via scikit-learn
16	NAB Machine Temperature	Anomaly detection	22,695 × 1	Numenta (NAB)
17	UCI Bank Marketing	Classification	45,211 × 15	OpenML 1461 / UCI
18	Adult Census Income	Classification	48,842 × 13	OpenML 1590 / UCI
19	Household Power Consumption	Forecasting	34,589 × 7 (hourly)	UCI (id 235)
20	Credit Card Fraud (ULB)	Anomaly, classification	284,807 × 30	OpenML 1597

4.3 SUPPORTED ALGORITHMS

The benchmark engine exposes 44 algorithm configurations (Table 4.2). Several classic algorithms exist in two versions: the from-scratch implementation written in Stage 2 and the equivalent scikit-learn model, so the two can be compared directly. Every task also includes at least one simple baseline.

Table 4.2: Algorithms available for each task

Task	No.	Algorithms
Regression	15	Mean baseline; Linear Regression (OLS, scratch normal-equation, scratch gradient-descent); Ridge; Lasso; Elastic Net; k-NN (scratch and sklearn); Decision Tree (scratch and sklearn); Random Forest; SVR; Gradient Boosting; XGBoost
Classification	13	Majority baseline; Logistic Regression (scratch and sklearn); k-NN (scratch and sklearn); Decision Tree (scratch and sklearn); Random Forest; SVM; Gaussian Naive Bayes; Gradient Boosting; XGBoost; MLP (PyTorch)
Clustering	4	K-Means; Agglomerative (Ward); Gaussian Mixture Model; DBSCAN
Forecasting	8	Naive; Seasonal naive; Linear AR; Random Forest and XGBoost on lag windows; LSTM; GRU; Temporal Transformer
Anomaly detection	4	Isolation Forest; Local Outlier Factor; One-Class SVM; Autoencoder (PyTorch)

4.4 INTELLIGENT DATASET SELECTION

When a user selects an algorithm, EPM lists only datasets whose task and input format suit it, ordered small-first for quick experiments or large-first for realistic benchmarks. When a user selects a dataset, it marks the *recommended* algorithms (fast, well-suited ones) and the full list of *compatible* ones. Incompatible combinations are refused with an explanation. For example, a forecasting model is never offered a shuffled tabular dataset, and the chatbot replies to "regression with logistic regression" by explaining that logistic regression is a classification algorithm.

4.5 LEAKAGE-FREE PREPROCESSING

Preprocessing is built as a scikit-learn **Pipeline** for each model and is fitted on the training split only (Figure 4.1). The same fitted pipeline is then applied, without refitting, to validation, test and future data, so training and inference always use identical transformations. Table 4.3 lists the steps.

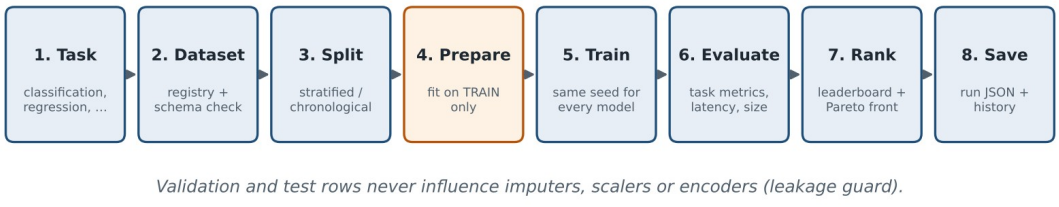


Figure 4.1: Experiment pipeline; preprocessing (step 4) is fitted on training rows only

Table 4.3: Preprocessing steps and when they are applied

Step	What it does	When it is applied
Duplicate removal	Drops repeated rows before splitting	Optional, on by default
Imputation	Median for numbers, most frequent value for categories	Always; statistics from training data
Scaling	Standardises numeric features	Only for scale-sensitive models (SVM, k-NN, linear, neural)
Encoding	One-hot encoding; unseen categories are safely ignored	When categorical columns exist
Outlier clipping	Winsorises at training-set quantiles	Optional
Class weighting	Balances rare classes during training	Optional, for imbalanced data
Windowing	Turns a time series into (past window → future horizon) samples	Forecasting and sequential anomaly tasks

Time-dependent data is always split chronologically, and windows are created after splitting so that no future value can appear in a training window. All preprocessing settings are saved with each run for reproducibility.

CHAPTER 5
BENCHMARKING AND EVALUATION

5.1 FAIR BENCHMARKING PROCEDURE

A benchmark compares several algorithms on one dataset. To keep the comparison fair, EPM uses one shared split and one random seed for every model, an optional k-fold cross-validation on the training data, and the same evaluation code for every model. For each model it records the task metrics, training time, inference latency per sample (including preprocessing), serialised model size and parameter count. Models are ranked by a primary metric, and a Pareto front marks the models that offer the best accuracy-speed trade-offs. Each run is saved as a JSON record with a configuration fingerprint so that it can be reopened or reproduced later.

5.2 EVALUATION METRICS

Table 5.1: Metrics used for each task

Task	Metrics reported	Primary (ranking) metric
Regression	MAE, MSE, RMSE, R^2	RMSE (lower is better)
Classification	Accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix	F1 (higher is better)
Clustering	Silhouette, Davies–Bouldin, Calinski–Harabasz; ARI against held-out labels when available	Silhouette (higher is better)
Forecasting	MAE, RMSE, MASE, sMAPE, comparison with naive baselines	RMSE (lower is better)
Anomaly detection	Precision, recall, F1, PR-AUC, ROC-AUC	PR-AUC (higher is better)

Metrics that cannot be computed in a given situation (for example ROC-AUC when the test set has a single class, or a silhouette score when DBSCAN finds only one cluster) are reported as "not defined" instead of a misleading number.

5.3 CLASSIFICATION RESULTS

Table 5.2 shows a ten-model benchmark on the Breast Cancer Wisconsin dataset (398 training and 114 test rows, stratified split, seed 42, 5-fold cross-validation). Figure 5.1 shows how a run appears in the Experiment Lab.

Table 5.2: Breast Cancer Wisconsin – classification benchmark

#	Model	F1	Acc.	CV F1 (mean ± std)	Latency (µs)	Size (KB)
1	Logistic Regression (scratch)	0.964	0.974	0.946 ± 0.049	24.8	12.7
2	Random Forest	0.963	0.974	0.928 ± 0.032	177.0	583.0
3	Logistic Regression (sklearn)	0.951	0.965	0.943 ± 0.051	22.6	3.8
4	Support Vector Machine (RBF)	0.951	0.965	0.955 ± 0.029	45.6	28.8
5	Gradient Boosting	0.950	0.965	0.930 ± 0.038	26.1	199.7

#	Model	F1	Acc.	CV F1 (mean ± std)	Latency (µs)	Size (KB)
6	k-NN (k = 5)	0.938	0.956	0.942 ± 0.043	42.8	100.0
7	XGBoost	0.937	0.956	0.940 ± 0.040	31.2	189.3
8	Gaussian Naive Bayes	0.909	0.939	0.916 ± 0.047	30.8	3.4
9	Decision Tree (depth 8)	0.905	0.930	0.875 ± 0.037	24.4	6.4
10	Majority-class baseline	0.000	0.632	0.000 ± 0.000	18.9	2.4

The from-scratch logistic regression written in Stage 2 matches the library models, which confirms that implementation. The top five models differ by less than the cross-validation standard deviation, so they should be treated as equally accurate; logistic regression is then the sensible choice because it is about 45 times smaller than the random forest and seven times faster. The majority baseline reaches 63% *accuracy* but an F1 of zero, showing why accuracy alone is misleading.

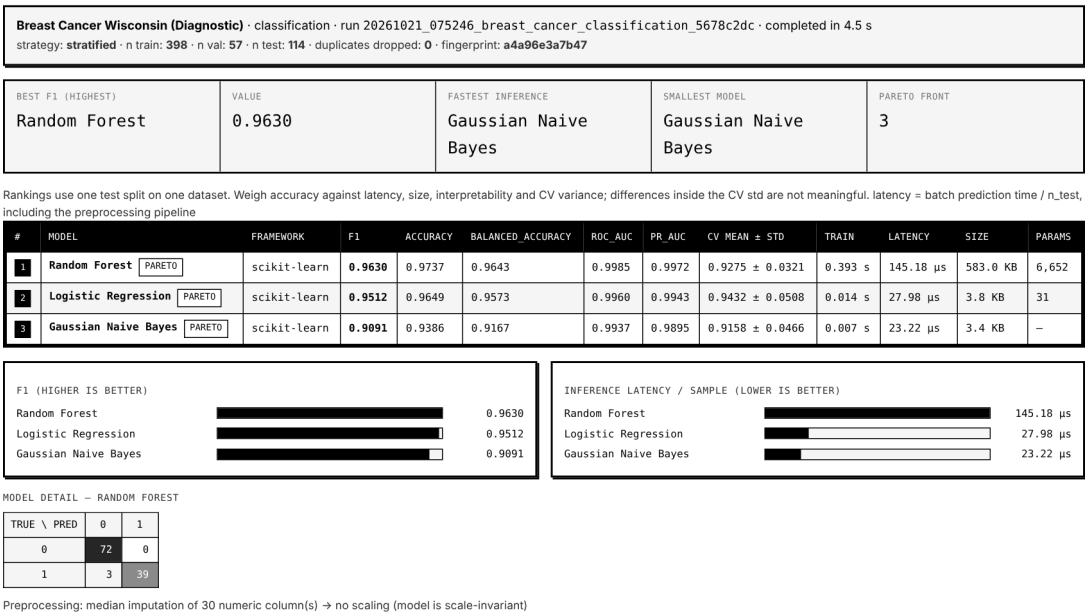


Figure 5.1: Experiment Lab result view with leaderboard, charts and confusion matrix

5.4 REGRESSION RESULTS

Table 5.3 shows the same procedure on the Diabetes dataset (308 training and 89 test rows, random split, 5-fold cross-validation).

Table 5.3: Diabetes progression – regression benchmark

#	Model	RMSE	R^2	CV RMSE (mean ± std)	Size (KB)
1	Gradient Boosting Regressor	52.80	0.474	57.47 ± 5.70	153.6
2	Support Vector Regression	52.99	0.470	58.93 ± 5.08	30.2
3	Random Forest Regressor	53.29	0.464	57.92 ± 4.74	909.4
4	XGBoost Regressor	53.32	0.463	59.49 ± 5.43	228.8
5	Ridge Regression	54.04	0.449	55.14 ± 4.12	2.2
6	Linear Regression (OLS and scratch)	54.06	0.448	55.14 ± 4.11	2.3

#	Model	RMSE	R^2	CV RMSE (mean ± std)	Size (KB)
8	Decision Tree (depth 8)	55.02	0.429	68.35 ± 6.14	4.4
9	k-NN Regressor	56.32	0.401	58.55 ± 4.24	34.4
11	Mean baseline	72.97	−0.005	78.48 ± 5.24	1.5

Gradient boosting wins on the single test split, but Ridge and linear regression have the best cross-validated error and are about 70 times smaller. This illustrates a key design decision in EPM: the leaderboard shows cross-validation spread next to the test score so that users do not over-trust one split.

5.5 RESULTS ON OTHER TASKS

Table 5.4: Best models on clustering, anomaly, forecasting and image tasks

Task / dataset	Best model (primary metric)	Observation
Clustering / Wine	K-Means (silhouette 0.285)	Also best agreement with true cultivars (ARI 0.897)
Clustering / Iris	DBSCAN (silhouette 0.597)	GMM scores lower silhouette but far better ARI (0.904 vs 0.555)
Clustering / Synthetic blobs	DBSCAN (silhouette 0.690)	K-Means, Ward and GMM recover the true clusters exactly (ARI 1.0)
Anomaly / NAB machine temp.	Isolation Forest (PR-AUC 0.781)	One-Class SVM 0.655; LOF 0.428; Autoencoder 0.367
Anomaly / Synthetic	Isolation Forest (PR-AUC 1.000)	Easy controlled case; all methods ≥ 0.997
Forecasting / Telemetry (1 step)	Naive persistence (RMSE 0.437, MASE 0.32)	Beat Linear AR (0.890), XGBoost (4.28), GRU (4.69), LSTM (6.44)
Classification / Digits 8×8	PyTorch MLP (F1 0.978)	SVM 0.975; Random Forest 0.972; Naive Bayes 0.813

Two lessons stand out. First, internal clustering scores and label agreement can disagree (Iris), so EPM reports both and never uses labels for training. Second, on short-horizon forecasting the simplest baseline beat every learned model, including the deep networks; without baselines and MASE this would have gone unnoticed.

CHAPTER 6
USER INTERFACE AND EPM CHATBOT

6.1 WEB DASHBOARD

Running `python run.py --serve` opens the EPM dashboard at http://localhost:3000. Its navigation bar leads to the project overview, methodology, a stage-by-stage Lab Workbench, the Experiment Lab, the Chat page, the leaderboards, the figures and a verification page that shows the live test status (Figure 6.1).

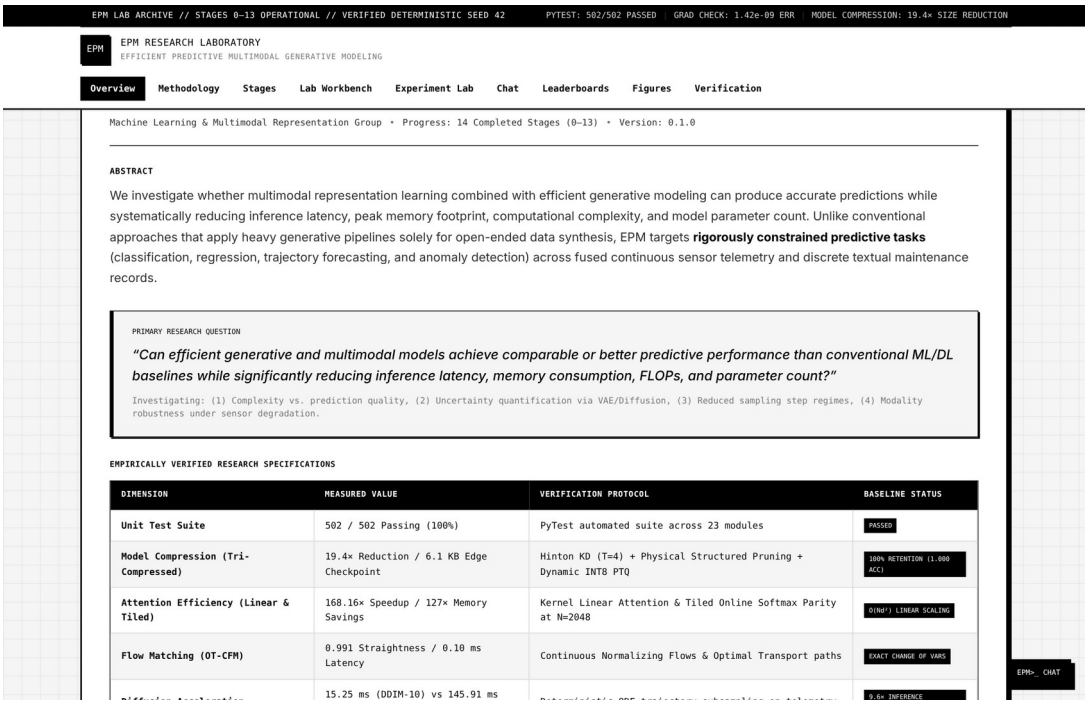


Figure 6.1: EPM dashboard overview page with the research question and verified results

6.2 EXPERIMENT LAB

The Experiment Lab guides the user through six numbered steps: (1) choose a task, (2) choose a dataset from the catalogue, which shows its source, size and download status, or upload a CSV file, (3) select algorithms using the Recommended or All compatible buttons, (4) adjust the configuration (seed, test fraction, cross-validation folds, scaling, class weighting, resource limits), (5) run the benchmark and watch the progress bar, and (6) reopen any earlier run from the experiment history (Figure 6.2).

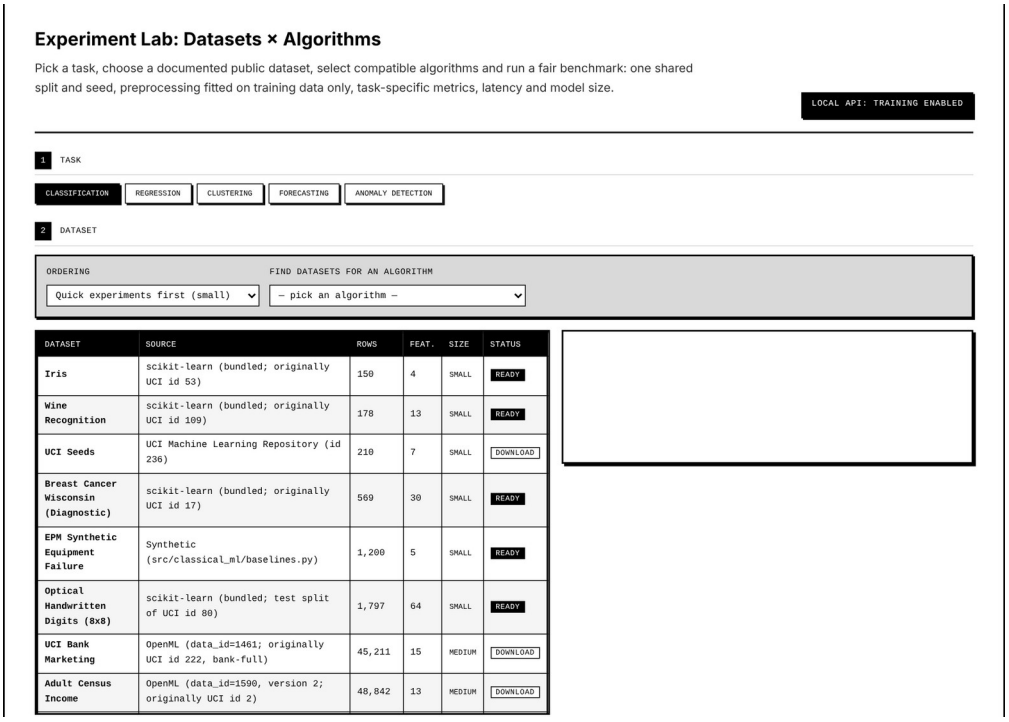


Figure 6.2: Experiment Lab: task tabs, dataset catalogue and algorithm finder

6.3 CHATBOT DESIGN

The EPM chatbot lets users run experiments in plain English, for example "classify iris with random forest" or "forecast the next 6 steps of sensor telemetry". It is fully offline and uses no large language model. Each message passes through three steps. **Intent recognition** uses ordered regular-expression rules, with a small TF-IDF and logistic-regression classifier as a fallback when no rule matches. **Slot extraction** finds the dataset, algorithms, task and settings by exact phrase matching, then regular expressions, then fuzzy matching for typing mistakes. A **state machine** (Figure 6.3) asks for missing information, confirms large jobs, runs them in the shared job queue and shows the results.

Table 6.1: Chatbot intents (25 in total)

Group	Intents	Example message
Run experiments	classify, regress, cluster, benchmark, train, forecast, detect_anomalies	"benchmark all algorithms on wine"
Generative and efficiency	generate, compare_generative, compress, uncertainty	"compare diffusion vs flow matching"
Explore	list_datasets, list_algorithms, describe_dataset, preview_data, download, recommend, explain	"what is the best model for diabetes?"
Results	show_results, compare	"compare knn and naive bayes"
Conversation	help, status, cancel, confirm, deny	"cancel the job"

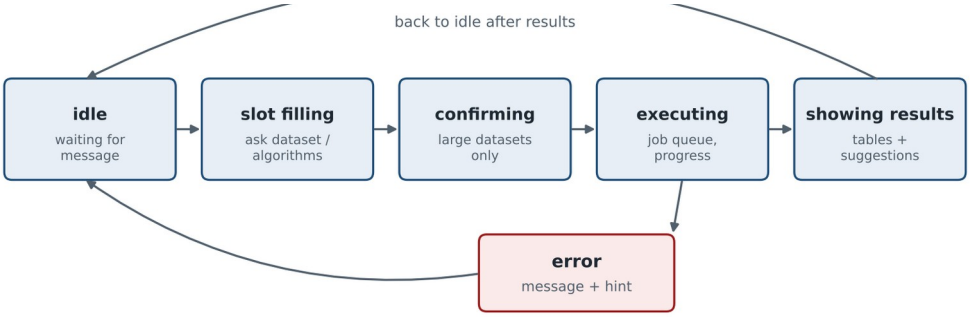


Figure 6.3: Conversation state machine of the EPM chatbot

Figure 6.4 shows a real conversation. The chatbot resolved three algorithm names, used the stratified split, reported the leaderboard with a Pareto marker, and offered follow-up suggestions such as "run again with 5-fold cv".

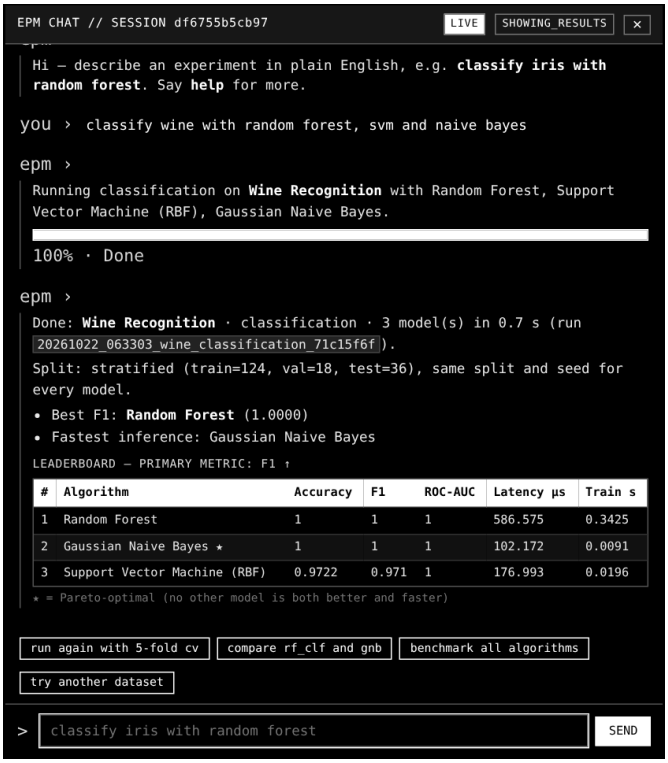


Figure 6.4: Chat panel running a classification benchmark on the Wine dataset

6.4 SAFETY GUARDRAILS

- The chatbot only calls existing EPM functions; it never executes arbitrary code.
- Inputs are validated: message length up to 1,000 characters, CV folds 2–10, epochs 1–300, horizon 1–168.
- At most five jobs can be queued, and any job can be cancelled at the next model boundary.
- Data previews are limited to 20 rows, and unknown dataset names are never replaced by a guess.

CHAPTER 7
TESTING AND RESEARCH RESULTS

7.1 AUTOMATED TESTING

The project contains 502 automated tests in 23 modules, and all 502 pass (`python run.py --test`). Table 7.1 groups them by area. The tests check, among other things, that preprocessing is fitted on training rows only, that chronological splits never place future rows in training, that metrics match hand-calculated values, that failed downloads and malformed files are handled gracefully, and that the same seed reproduces the same results.

Table 7.1: Summary of the automated test suite

Area	Test modules	Tests
Foundations	math_foundations, config, reproducibility, experiment	111
Chatbot and API	chatbot, local_api	110
Datasets and preprocessing	datasets, preprocessing_pipelines	70
Benchmarking and metrics	benchmarking, task_metrics, metrics	66
Efficiency, compression, research	efficiency, compression, research	44
Classical, neural, NLP	classical_ml, neural_networks, nlp	39
Generative models	generative, diffusion, flow_matching	34
Time series and multimodal	timeseries, multimodal, timeseries_diffusion	28
Total	**23 modules**	**502 (100% pass)**

7.2 STAGE 13: REDUCED-STEP GENERATIVE FORECASTING

The research hypothesis **H1** states that a diffusion forecaster sampled with fewer steps can keep competitive accuracy while greatly reducing inference cost. To test only the effect of the step count, the *same* trained network was sampled with 100, 50, 20, 10 and 5 DDIM steps on synthetic industrial telemetry (24-step window, 12-step horizon, chronological split, two seeds, 95% bootstrap confidence intervals). Real baselines (linear autoregression, LSTM, Transformer) were trained on the same split.

Table 7.2: Stage 13 efficiency analysis (reference: 100 steps)

Steps (NFE)	RMSE	Latency (ms)	Speed-up	Change in RMSE	Within full-step CI?
50	2.75	100.9	2.09×	−2.5%	Yes
20	2.88	43.1	4.89×	+2.0%	Yes
10	3.00	27.5	7.66×	+6.2%	Yes
5	5.20	14.6	14.4×	+84.1%	No

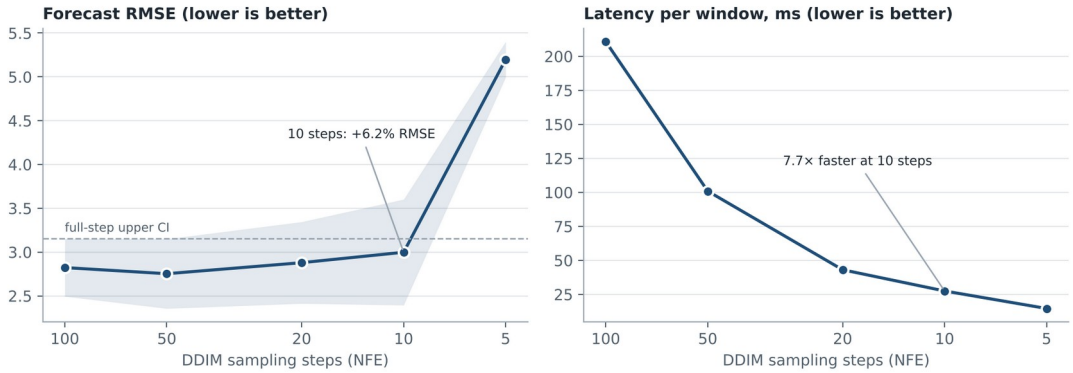


Figure 7.1: Accuracy and latency of one diffusion model at different step counts

The reference 100-step sampler reached an RMSE of 2.82 at 210.8 ms per window. With 10 steps the model is 7.66 times faster and its error stays inside the confidence interval of the full 100-step sampler, which supports H1 on this dataset. At 5 steps the error rises by 84%, which marks the limit. The diffusion model (RMSE ≈ 2.8) was also more accurate than the trained linear (4.53), LSTM (7.50) and Transformer (8.49) baselines, and additionally provides prediction intervals.

7.3 FINDINGS AND LIMITATIONS

- Simple baselines are essential: on one-step telemetry forecasting, "repeat the last value" beat all learned models.
- Clipping the denoised estimate (an optional setting added to the diffusion sampler) reduced a sampling blow-up on sensor telemetry from a sliced-Wasserstein distance of 146 to 0.149.
- A VAE with the default weighting collapsed to near-constant samples on small tables; a corrected weighting ($\beta = 0.5/d$) is used and reported.
- The Stage 13 study uses synthetic telemetry and CPU timings; results on real sensors and GPUs may differ.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE OF WORK

8.1 CONCLUSION

EPM has grown from a series of learning stages into a reliable and reproducible machine-learning experimentation platform. Twenty documented datasets and 44 algorithm configurations covering five task families are connected through a central registry, compatibility rules, leakage-free preprocessing pipelines and a single fair benchmarking protocol. Results are presented with task-appropriate metrics, cross-validation spread, latency, model size and a Pareto front, so that users can choose models sensibly rather than by accuracy alone. The Experiment Lab and the offline chatbot make correct experiments accessible to beginners, and 502 passing tests verify the behaviour. The Stage 13 study showed that a diffusion forecaster can run 7.66 times faster with 10 sampling steps while staying within the accuracy confidence interval of the 100-step sampler.

8.2 FUTURE SCOPE

- Add image datasets such as Fashion-MNIST and CIFAR-10 to the benchmark runner for CNN models.
- Integrate NASA C-MAPSS turbofan data for remaining-useful-life prediction.
- Add rolling-origin cross-validation for every forecasting model and ARIMA/SARIMA baselines.
- Provide a GPU-backed hosted version so that experiments can be run from the public website.
- Add fairness metrics for the Adult and Bank Marketing datasets.
- Repeat the Stage 13 study on real industrial sensor archives.

APPENDIX A: HOW TO RUN THE PROJECT

Step 1 – Get the code. Install Python 3.10 or newer and Git, then run:

```
git clone https://github.com/webex042/effective-prediction-model.git
cd effective-prediction-model
```

Step 2 – Create an environment and install the dependencies.

```
python -m venv venv
# Windows:      venv\Scripts\activate
# macOS/Linux:  source venv/bin/activate
pip install -r requirements.txt
```

Step 3 – Use the platform.

```
# open the dashboard at http://localhost:3000
python run.py --serve
# chat with EPM in the terminal
python run.py --chat
# benchmark one dataset from the command line
python run.py --benchmark breast_cancer --cv 5
# run all 502 tests / check datasets / run a stage demo
python run.py --test
python run.py --verify-datasets
python run.py --stage 13
```

Step 4 – Read more. Each concept has its own guide in the docs folder of the repository, and the README lists every command and folder.

Table A.1: Useful example chat messages

You type	EPM does
classify iris with random forest	Trains and evaluates Random Forest on Iris and shows the leaderboard
benchmark all algorithms on diabetes	Runs every compatible regression model with the same split
forecast sensor telemetry 6 steps ahead	Runs forecasting models with a chronological split and baselines
describe adult	Shows source, licence, size, target and caveats of the dataset
what is flow matching?	Answers with an excerpt from the project documentation

REFERENCES

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[2] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[4] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," ACM Trans. Knowledge Discovery from Data, vol. 6, no. 4, 2012.

[5] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," International Journal of Forecasting, vol. 22, no. 4, pp. 679–688, 2006.

[6] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.

[7] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[8] A. Lavin and S. Ahmad, "Evaluating Real-Time Anomaly Detection Algorithms – The Numenta Anomaly Benchmark," in Proc. IEEE ICMLA, 2015.

[9] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in Int. Conf. Learning Representations (ICLR), 2014.

[10] J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[11] J. Song, C. Meng and S. Ermon, "Denoising Diffusion Implicit Models," in Int. Conf. Learning Representations (ICLR), 2021.

[12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, "Flow Matching for Generative Modeling," in Int. Conf. Learning Representations (ICLR), 2023.

[13] A. Tong et al., "Improving and generalizing flow-based generative models with minibatch optimal transport," Transactions on Machine Learning Research, 2024.

[14] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[15] G. Hinton, O. Vinyals and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.

[16] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in Proc. Int. Conf. Machine Learning (ICML), 2017.

[17] M. Kelly, R. Longjohn and K. Nottingham, "The UCI Machine Learning Repository," University of California, Irvine. [Online]. Available: https://archive.ics.uci.edu

[18] J. Vanschoren, J. N. van Rijn, B. Bischl and L. Torgo, "OpenML: Networked Science in Machine Learning," SIGKDD Explorations, vol. 15, no. 2, pp. 49–60, 2013.

[19] EPM project repository and documentation, GitHub. [Online]. Available: https://github.com/webex042/effective-prediction-model